

Lecture 2: Computation — Julia and VFI

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 12, 2026

University of Toronto

Setup

Bellman Equation

$$V(b, z) = \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + z, z) \quad \forall b, z$$

b is wealth. z is the agent's productivity (constant for now).

This equation characterizes the function V that is needed to solve the household's problem.

V is Unique and Computable

For any bounded v_1, v_2 ,

$$\|\mathcal{T}v_1 - \mathcal{T}v_2\|_\infty \leq \beta \|v_1 - v_2\|_\infty.$$

Informally, the Banach fixed-point theorem implies:

- A unique V^* exists that satisfies $V^* = \mathcal{T}V^*$.
- Value function iteration will always find V^* .

This lecture: replace the continuous state b with a finite grid, code up $\mathcal{T}$ as a Julia function, iterate to the fixed point, plot V .

Julia

Arrays

```
xs = [1.0, 2.0, 3.0]      # Vector{Float64}, length 3
M  = [1 2; 3 4]           # Matrix{Int64}, 2×2
zs = zeros(3, 4)          # 3×4 matrix of 0.0
xs[1]                     # 1.0 --- 1-indexed!
M[2, 1]                   # 3
```

- **Indexing starts at 1.**
- `xs[end]` is the last element. `xs[2:3]` is a slice.
- Arrays are column-major: dimension 1 is the row, dimension 2 is the column. A “vector” is a column vector.

Broadcasting

```
xs = [1.0, 2.0, 3.0]

xs .+ 10      # [11.0, 12.0, 13.0]
log.(xs)      # [0.0, 0.693, 1.099]
xs .^ 2       # [1.0, 4.0, 9.0]

# elementwise on two arrays of the same shape:
[1,2,3] .* [10,20,30] # [10, 40, 90]
```

The dot `.` is the “elementwise” marker.

Dots apply any function or operation elementwise with no need for loops.

Functions

```
function utility(c)
    c <= 0 && return -Inf
    return log(c)
end

# shorthand for simple conditionals:
function utility(c)
    return c <= 0 ? -Inf : log(c)
end

# one-line functions equally valid:
u(c) = c <= 0 ? -Inf : log(c)
```

Discretization

Discrete Wealth Grid

- V is a function on $[0, \infty) \times W$. We can't represent it exactly.
- Pick N wealth values $b_1 < b_2 < \dots < b_N$.
- Represent V as the vector $[V(b_1), \dots, V(b_N)]$.
- This makes the function finite-dimensional.

Note: None of the concepts of value function iteration **depend** on V being represented as a vector. V could be a spline or a neural network or over a dynamic grid and the concepts would still be valid.

Linear vs Log Spacing

```
loggrid(lo, hi, N) = exp.(range(log(lo), log(hi); length=N))

# linear: equally spaced
b_grid = collect(range(0.1, 20.0; length=200))

# log: denser near the borrowing constraint
b_grid = loggrid(0.1, 20.0, 200)
```

The policy function is more nonlinear for small b , so we use log spacing to put more grid points on smaller b .

Utility

$$u(c) = c \leq 0 ? -\text{Inf} : \log(c)$$

Generalized version is CRRA with γ , but keep it simple for now.

Bellman Operator $\mathcal{T}$ as a Julia Function

```
function bellman_operator(V, params)
    # Unpack parameters from `params`
    (; β, R, z, b_grid) = params

    # Compute saving levels `b_end` necessary to achieve each level of next-period wealth `b_next`
    b_end = (b_grid' .- z)./R

    # Maximize conditional utility `u.(b_grid .- b_end) .+ β.*V'` across the `b_next` dimension (dimension 2,
    # columns).
    return maximum(u.(b_grid .- b_end) .+ β.*V'; dims=2) |> vec
end
```

`b_grid` is a column vector. `b_grid'` is a row vector.

Each row corresponds to a b value; each column to a b' value. The last line takes max **within** each b (rows), **across** b' (columns).

Note about Examples

The code in the notebooks are **not** optimized or fast. They are just intended to illustrate the concepts.

I provide much faster implementations in the package `HouseholdStages.jl`.

Value Function Iteration

Value Function Iteration

1. Start with $V_0 \equiv 0$ (any initial guess works).
2. $V_{k+1} \leftarrow \mathcal{T}V_k$.
3. Stop when $\|V_{k+1} - V_k\|_\infty < \text{tol}$.

solve_vfi

```
function solve_vfi(params; rtol=1e-6, verbosity=0)
    # Make initial guess of V
    V = zeros(size(params.b_grid))

    # Repeatedly apply Bellman operator until convergence
    Δ = Inf
    while Δ >= rtol
        # Update V
        TV = bellman_operator(V, params)

        # Calculate error
        Δ = maximum(abs.(TV .- V))

        # Update V
        V = TV
    end

    return V
end
```

Baseline Calibration

```
b_grid = loggrid(0.05, 40.0, 400)
params = β(β=0.96, R=1.04, z=1.0, b_grid)

V = solve_vfi(params)
```

- $\beta R < 1$, otherwise V^* is infinite.
- Grid: 400 log-spaced points from 0.05 to 40.
- params is a NamedTuple (named tuple). The last b_grid is equivalent to b_grid=b_grid.

Features of Value Function

The shape of V is similar to the shape of u^1 ,

- $V(b)$ is increasing (more wealth $\Rightarrow$ more future consumption).
- $V(b)$ is concave: an additional unit of wealth is worth more to the poor than to the rich.
- The slope is the **marginal value of wealth**: $V'(b) = u'(c^*(b))$ by the envelope theorem.

¹Sheeran (2017)

Policy Function c^*

$$c^*(b) = \arg \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + z).$$

```
function policy_function(V, params)
    (; β, R, z, b_grid) = params

    # same setup as bellman_operator
    b_end = (b_grid' .- z) ./ R

    # argmax over b' (columns) – pull out the column index for each row
    idx = argmax(u.(b_grid .- b_end) .+ β .* V'; dims=2)
    return vec(getindex.(idx, 2))::Vector{Int}
end
```

Same as `bellman_operator`, except with `argmax` instead of `maximum`.

Returns a vector of integers: grid index of the optimal b' for each (row) b .